Efficient String Usage in Low-Resource Embedded Systems

Introduction

When developing for resource-constrained environments like Arduino, ESP8266, or other embedded systems with limited RAM and ROM, efficient string handling is crucial. This document demonstrates best practices for string manipulation in C/C++ that minimize memory usage while maintaining functionality.

Understanding Memory Constraints

- **Arduino Uno**: 2KB RAM, 32KB Flash
- **ESP8266**: 80KB RAM, 4MB Flash (but limited heap)
- **ESP32**: 520KB RAM, 16MB Flash

Key considerations:

- Avoid dynamic memory allocation when possible
- Use stack-based strings for small, temporary data
- Minimize string copying
- Use PROGMEM for constant strings on AVR platforms
- Prefer fixed-size buffers over dynamic strings

Best Practices for String Handling

1. Use Fixed-Size Character Arrays Instead of std::string

```
// Bad - uses dynamic memory (not available on Arduino)
std::string message = "Hello";
message += " World";

// Good - fixed size, stack-based
char message[20];
strncpy(message, "Hello", sizeof(message));
strncat(message, " World", sizeof(message) - strlen(message) - 1);
```

2. Avoid Unnecessary String Copying

```
// Bad - multiple copies
String processMessage(String input) {
    String result = input;
    result += " processed";
    return result;  // Creates another copy
}

// Good - modify in place
void processMessage(char* buffer, size_t bufferSize) {
    size_t len = strlen(buffer);
    if (len + 11 < bufferSize) {  // " processed" = 10 chars + null
        strcat(buffer, " processed");
    }
}
```

3. Use PROGMEM for Constant Strings (AVR Platforms)

```
#include <avr/pgmspace.h>

// Store constant strings in flash memory
const char message1[] PROGMEM = "Temperature: ";
const char message2[] PROGMEM = "Humidity: ";
const char message3[] PROGMEM = "Pressure: ";

// Function to read from PROGMEM
void printFromProgmem(const char* progmemString) {
    char buffer[20];
    strcpy_P(buffer, progmemString);
    Serial.print(buffer);
}

// Usage
printFromProgmem(message1);
Serial.println(temperature);
```

4. Implement Efficient String Operations

```
// Efficient string length calculation
size_t stringLength(const char* str) {
    const char* s = str;
    while (*s) s++;
    return s - str;
}

// Safe string copy with bounds checking
void safeCopy(char* dest, const char* src, size_t destSize) {
    if (destSize == 0) return;
    size_t i;
    for (i = 0; i < destSize - 1 && src[i]; i++) {
        dest[i] = src[i];
    }
    dest[i] = '\0';
}

// Efficient string comparison
int efficientCompare(const char* str1, const char* str2) {
    while (*str1 && *str2 && *str1 == *str2) {
        str1++;
        str2++;
    }
    return *str1 - *str2;
}
```

5. Use Buffer Pools for Repeated Operations

```
#define BUFFER_POOL_SIZE 3
#define BUFFER_SIZE 64

char bufferPool[BUFFER_POOL_SIZE][BUFFER_SIZE];
bool bufferInUse[BUFFER_POOL_SIZE] = {false};

char* getBuffer() {
    for (int i = 0; i < BUFFER_POOL_SIZE; i++) {
        if (!bufferInUse[i]) {
            bufferInUse[i] = true;
            return bufferPool[i];
        }
    }
    return nullptr;  // No free buffers
}

void releaseBuffer(char* buffer) {
    for (int i = 0; i < BUFFER_POOL_SIZE; i++) {
        if (buffer == bufferPool[i]) {
            bufferInUse[i] = false;
            return;
        }
    }
}
```

6. Minimize String Concatenation

```
// Bad - multiple allocations
String buildMessage(int temp, int humidity) {
    String msg = "Temp: ";
    msg += String(temp);
    msg += "C, Humidity: ";
    msg += String(humidity);
    msg += "%";
    return msg;
}

// Good - single buffer operation
void buildMessage(char* buffer, size_t bufferSize, int temp, int humidity) {
    snprintf(buffer, bufferSize, "Temp: %dC, Humidity: %d%%", temp, humidity);
}
```

7. Use Flash String Helper (FSH) on ESP8266/ESP32

```
// For ESP8266/ESP32 - store strings in flash
const char MESSAGE_TEMP[] PROGMEM = "Temperature: %d C";
const char MESSAGE_HUMID[] PROGMEM = "Humidity: %d %%";
const char MESSAGE_ERROR[] PROGMEM = "Sensor error";

// Efficient printing without RAM usage
void printSensorData(int temp, int humidity) {
    char buffer[32];
    snprintf_P(buffer, sizeof(buffer), MESSAGE_TEMP, temp);
    Serial.println(buffer);

    snprintf_P(buffer, sizeof(buffer), MESSAGE_HUMID, humidity);
    Serial.println(buffer);
}
```

8. Implement Custom Lightweight String Class

```
class LightweightString {
private:
    char* buffer;
    size_t capacity;
    size_t length;

public:
    LightweightString(size_t cap) : capacity(cap), length(0) {
        buffer = new char[cap];
        buffer[0] = '\0';
    }

    ~LightweightString() {
        delete[] buffer;
    }

    bool append(const char* str) {
        size_t strLen = strlen(str);
        if (length + strLen >= capacity) return false;

        strcpy(buffer + length, str);
        length += strLen;
        return true;
    }

    const char* c_str() const { return buffer; }
    size_t size() const { return length; }
    void clear() {
        buffer[0] = '\0';
        length = 0;
    }
};
```

9. Optimize String Parsing

```
// Efficient CSV parsing
struct SensorData {
    int temperature;
    int humidity;
};

bool parseSensorData(const char* csvLine, SensorData* data) {
    char tempStr[10];
    char humidStr[10];
    int tempIndex = 0;
    int humidIndex = 0;
    bool isHumidity = false;

    for (const char* p = csvLine; *p && tempIndex < 9 && humidIndex < 9; p++) {
        if (*p == ',') {
            isHumidity = true;
            continue;
        }
        if (!isHumidity) {
            tempStr[tempIndex++] = *p;
        } else {
            humidStr[humidIndex++] = *p;
        }
    }

    tempStr[tempIndex] = '\0';
    humidStr[humidIndex] = '\0';

    data->temperature = atoi(tempStr);
    data->humidity = atoi(humidStr);

    return true;
}
```

10. Memory-Efficient JSON Handling

```
// Lightweight JSON parser for sensor data
struct JsonParser {
    char* buffer;
    size_t bufferSize;

    JsonParser(char* buf, size_t size) : buffer(buf), bufferSize(size) {}

    bool parseSensorJson(const char* json, int* temp, int* humidity) {
        // Simple JSON parser for {"temp":25,"humidity":60}
        const char* tempKey = "\"temp\":";
        const char* humidKey = "\"humidity\":";

        const char* tempPos = strstr(json, tempKey);
        const char* humidPos = strstr(json, humidKey);

        if (!tempPos || !humidPos) return false;

        *temp = atoi(tempPos + strlen(tempKey));
        *humidity = atoi(humidPos + strlen(humidKey));

        return true;
    }
};
```

Complete Example: Weather Station

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

// Configuration
#define WIFI_SSID "your_ssid"
#define WIFI_PASSWORD "your_password"
#define SERVER_URL "http://api.example.com/data"

// Memory-efficient string buffers
char jsonBuffer[128];
char responseBuffer[256];
char logBuffer[64];

void setup() {
    Serial.begin(115200);

    // Connect to WiFi
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("WiFi connected");
}

void loop() {
    // Read sensors (simulated)
    int temperature = random(20, 30);
    int humidity = random(40, 80);

    // Build JSON efficiently
    snprintf(jsonBuffer, sizeof(jsonBuffer),
             "{\"temperature\":%d,\"humidity\":%d}", temperature, humidity);

    // Send data
    if (sendSensorData(jsonBuffer)) {
        snprintf(logBuffer, sizeof(logBuffer), "Data sent: T=%d, H=%d", temperature, humidity);
        Serial.println(logBuffer);
    } else {
        Serial.println("Failed to send data");
    }

    delay(60000);  // Send every minute
}

bool sendSensorData(const char* jsonData) {
    if (WiFi.status() != WL_CONNECTED) return false;
```

```
    HTTPClient http;
    http.begin(SERVER_URL);
    http.addHeader("Content-Type", "application/json");

    int httpResponseCode = http.POST(jsonData);

    if (httpResponseCode > 0) {
        // Read response efficiently
        size_t responseSize = http.getSize();
        if (responseSize > 0 && responseSize < sizeof(responseBuffer)) {
            http.getString().toCharArray(responseBuffer, sizeof(responseBuffer));
            Serial.print("Response: ");
            Serial.println(responseBuffer);
        }
        http.end();
        return httpResponseCode == 200;
    }

    http.end();
    return false;
}
```

Memory Optimization Techniques

1. Use Static Buffers

```
// Global static buffers reduce stack usage
static char globalBuffer[128];

void processData() {
    // Use globalBuffer instead of local arrays
    strcpy(globalBuffer, "Processing data...");
    // ... process ...
}
```

2. Avoid Recursion

```
// Bad - recursion uses stack space
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

// Good - iterative approach
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

3. Use Bit Fields for Flags

```
// Memory-efficient flags
struct SensorFlags {
    bool temperatureValid : 1;
    bool humidityValid : 1;
    bool pressureValid : 1;
    bool errorOccurred : 1;
    // 4 bits instead of 4 bytes
};
```

4. Optimize Data Structures

```
// Use unions to save space for mutually exclusive data
union SensorValue {
    int intValue;
    float floatValue;
    char stringValue[10];
};

struct SensorReading {
    char sensorId[5];
    SensorValue value;
    SensorFlags flags;
    // Total: ~15 bytes instead of ~25+ with separate fields
};
```

Performance Benchmarking

```
// Simple benchmarking function
unsigned long benchmarkStringOperation() {
    char buffer[64];
    unsigned long start = millis();

    for (int i = 0; i < 1000; i++) {
        snprintf(buffer, sizeof(buffer), "Iteration %d", i);
        // Simulate processing
        volatile int dummy = strlen(buffer);
        (void)dummy;
    }

    return millis() - start;
}

void printMemoryUsage() {
    // For ESP8266/ESP32
    Serial.printf("Free heap: %d bytes\n", ESP.getFreeHeap());
    Serial.printf("Heap fragmentation: %d%%\n", ESP.getHeapFragmentation());
}
```

Best Practices Summary

1. **Use fixed-size buffers** instead of dynamic strings
2. **Store constants in PROGMEM** on AVR platforms
3. **Minimize string copying** and concatenation
4. **Use stack-based buffers** for temporary operations
5. **Implement bounds checking** to prevent buffer overflows
6. **Pool reusable buffers** to reduce allocations
7. **Use efficient parsing** techniques
8. **Profile memory usage** regularly
9. **Avoid unnecessary features** from standard libraries
10. **Test on target hardware** early and often

Following these practices will help you create efficient, reliable embedded applications that make the most of limited resources while maintaining code readability and maintainability.